

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

КУРСОВАЯ РАБОТА (ПРОЕКТ)

по дисциплине «Проектирование и архитектура программных систем»
наименование дисциплины

на тему Моделирование продукционной системы Маркова

Направление подготовки (специальность) 09.03.04
(код, наименование)

Программная инженерия

Автор работы (проекта) Алехин А. А.
(инициалы, фамилия) (подпись, дата)

Группа ПО-226

Руководитель работы (проекта) А. А. Чаплыгин
(инициалы, фамилия) (подпись, дата)

Работа (проект) защищена
(дата)

Оценка

Члены комиссии

подпись, дата	фамилия и. о.
подпись, дата	фамилия и. о.
подпись, дата	фамилия и. о.

Курск 2025 г.

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

ЗАДАНИЕ НА КУРСОВУЮ РАБОТУ (ПРОЕКТ)

Студента Алехина А.А., шифр 22-06-0196, группа ПО-226

1. Тема «Моделирование продукционной системы Маркова» .

2. Срок предоставления работы к защите «31» января 2025 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) проанализировать теоретические основы продукционных систем Маркова и их алгоритмическое поведение;

2) разработать концептуальную модель программной системы для моделирования продукционной системы Маркова;

3) спроектировать программную систему моделирования продукционной системы Маркова с графическим интерфейсом (Tkinter/CustomTkinter);

4) сконструировать, реализовать и протестировать программную систему моделирования продукционной системы Маркова на Python.

3.2. Входные данные и требуемые результаты для программы:

1) Входными данными для программной системы являются: набор правил продукционной системы Маркова; исходная строка; конфигурация начального состояния.

2) Выходными данными для программной системы являются: пошаговое выполнение правил системы; промежуточные состояния строки; финальный результат применения правил; лог работы; визуализация процесса работы системы.

4. Содержание работы (по разделам):

4.1. Введение.

4.2. Анализ предметной области: теоретические основы продукционных систем Маркова, формальное определение, правила работы, примеры алгоритмов.

4.3. Техническое задание: основание для разработки, назначение разработки, требования к программной системе моделирования продукционной системы Маркова, требования к оформлению документации.

4.4. Технический проект: общие сведения о программной системе, проект данных программной системы, проектирование архитектуры программной системы, проектирование пользовательского интерфейса (Tkinter/CustomTkinter).

4.5. Рабочий проект: спецификация компонентов и классов программной системы, тестирование программной системы, сборка компонентов программной системы.

4.6. Заключение.

4.7. Список использованных источников.

5. Перечень графического материала:

Руководитель работы (проекта)	_____	_____
	(подпись, дата)	А. А. Чаплыгин (инициалы, фамилия)
Задание принял к исполнению	_____	_____
	(подпись, дата)	Алехин А. А. (инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 51 странице. Работа содержит 9 иллюстраций, 10 таблиц, 10 библиографических источников и 0 листов графического материала. Количество приложений – 1. Фрагменты исходного кода представлены в приложении А.

Перечень ключевых слов: производственная система Маркова, моделирование, алгоритм, правила замены, логирование, статистика, Python, Tkinter, CustomTkinter, графический интерфейс, пошаговое выполнение, анимация, визуализация, строка, правило, левый и правый символ, конфигурация.

Объектом разработки является программная система на языке Python для симуляции работы производственной системы Маркова с графическим интерфейсом и возможностью визуализации пошагового выполнения алгоритмов.

Целью работы является создание программной реализации производственной системы Маркова для демонстрации принципов работы формальных алгоритмических систем, изучения применения правил замены и визуального анализа выполнения алгоритмов.

В процессе разработки были реализованы основные компоненты системы: правила Маркова (левая и правая части), строка для преобразования, механизм применения правил, подсчет статистики выполнения (количество шагов, частота применения правил, длина строки на каждом шаге), журнал логирования всех шагов, а также графический интерфейс для визуализации процесса с использованием Tkinter и CustomTkinter.

Разработанное приложение позволяет загружать правила из текстовых файлов, вводить правила вручную, выполнять алгоритм пошагово с анимацией, сохранять журнал работы и отображать статистику. Программная реализация успешно демонстрирует работу производственной системы Маркова на различных примерах алгоритмов.

ABSTRACT

The volume of work is 51 page. The work contains 9 illustrations, 10 tables, 10 bibliographic sources and 0 sheets of graphic material. The number of applications is 2. The graphic material is presented in annex A. Fragments of the source code are presented in annex B.

List of keywords: Markov production system, modeling, algorithm, replacement rules, logging, statistics, Python, Tkinter, CustomTkinter, graphical user interface, step-by-step execution, animation, visualization, string, rule, left and right parts, configuration.

The object of development is a software system implemented in Python for simulating a Markov production system with a graphical interface and the ability to visualize step-by-step algorithm execution.

The purpose of the work is to create a software implementation of a Markov production system to demonstrate the principles of formal algorithmic systems, study the application of replacement rules, and perform visual analysis of algorithm execution.

During development, the main components of the system were implemented: Markov rules (left and right parts), the working string, the rule application mechanism, execution statistics (number of steps, frequency of rule application, string length at each step), logging of all steps, and a graphical interface for visualizing the process using Tkinter and CustomTkinter.

The developed application allows loading rules from text files, manual rule input, step-by-step execution with animation, saving execution logs, and displaying statistics. The software implementation successfully demonstrates the operation of a Markov production system on various algorithm examples.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	9
1 Анализ предметной области	12
1.1 Продукционные системы Маркова: история и значение	12
1.2 Формальное определение и структура системы Маркова	12
1.3 Механизм функционирования и пример работы	13
1.4 Цели и задачи моделирования системы Маркова	13
1.5 Существующие решения и их ограничения	14
1.6 Выводы по предметной области	14
2 Техническое задание	15
2.1 Основание для разработки	15
2.2 Цель и назначение разработки	15
2.3 Требования пользователя к интерфейсу приложения	16
2.4 Моделирование вариантов использования	18
2.5 Описание сценария использования	19
2.6 Требования к оформлению документации	20
3 Технический проект	21
3.1 Общая характеристика организации решения задачи	21
3.2 Обоснование выбора технологии проектирования	21
3.2.1 Описание используемых технологий и языков программирования	21
3.2.2 Язык программирования Python	21
3.2.2.1 Достоинства языка Python	21
3.2.2.2 Недостатки языка Python	22
3.2.3 Библиотека CustomTkinter	22
3.2.3.1 Достоинства CustomTkinter	22
3.2.3.2 Недостатки CustomTkinter	22
3.3 Диаграмма компонентов и схема обмена данными между файлами компонента	22
3.4 Диаграмма размещения	24

3.5	Содержание информационных блоков. Основные сущности	24
3.5.0.1	Описание сущностей и их атрибутов	25
4	Рабочий проект	27
4.1	Классы, используемые при разработке приложения	27
4.2	Модульное тестирование	29
4.3	Системное тестирование	30
4.4	Графический интерфейс и анимация работы алгоритма	30
4.5	Отображение статистики и журнала шагов	32
4.6	Особенности реализации	33
	ЗАКЛЮЧЕНИЕ	35
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36
	Приложение А — Исходный код программы	38

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

ПСМ – производственная система Маркова.

Правило Маркова – кортеж вида (левая часть, правая часть, завершающее правило), определяющий замену подстроки.

Строка – последовательность символов, обрабатываемая системой.

Лог – файл или структура данных, фиксирующая все шаги преобразования строки.

Статистика – набор данных о работе системы: количество шагов, частота применения правил, длина строки на каждом шаге.

GUI – графический пользовательский интерфейс.

Tkinter – стандартная библиотека Python для создания графических интерфейсов.

CustomTkinter – расширение Tkinter для современного оформления интерфейсов на Python.

Python – высокоуровневый язык программирования, на котором реализована система.

Анимация – пошаговая визуализация процесса применения правил.

JSON (JavaScript Object Notation) – текстовый формат представления структурированных данных.

ТЗ – техническое задание.

ТП – технический проект.

ВВЕДЕНИЕ

Формальные модели вычислений, к которым относится продукционная система Маркова, занимают важное место в теории алгоритмов и математической логике. Такие системы служат базисом для понимания фундаментальных принципов обработки информации, построения формальных языков и доказательства вычислимости различных функций. Разработанная Андреем Андреевичем Марковым в середине XX века нормальная алгоритмическая система (или система Маркова) стала одной из первых формальных моделей вычислений, эквивалентных машине Тьюринга по вычислительной мощности.

Продукционная система Маркова представляет собой совокупность правил переписывания, последовательно применяемых к строке символов. Алгоритм функционирует на основе строгой последовательности замены подстрок в тексте, пока не будет достигнуто завершающее состояние. Несмотря на внешнюю простоту, такие системы позволяют описывать и моделировать широкий класс алгоритмов, а также служат хорошим инструментом для изучения понятий вычислимости и формальных грамматик.

Практическое значение моделирования системы Маркова заключается в возможности демонстрации процессов работы формальных алгоритмов, обучения студентов принципам функционирования систем подстановок, а также проверки корректности и полноты наборов правил. Программная реализация симулятора системы Маркова позволяет не только пошагово проследить выполнение алгоритма, но и собрать статистику его работы, включая количество применённых правил и изменение длины строки на каждом этапе.

Современные средства разработки предоставляют широкие возможности для визуализации работы подобных систем. Использование языка программирования Python в сочетании с библиотеками tkinter и customtkinter позволяет создать наглядный и удобный графический интерфейс, обеспечивающий простоту взаимодействия с пользователем. В разработанной програм-

ме реализованы функции пошагового выполнения, автоматического запуска алгоритма, сохранения логов преобразований и вывода статистики выполнения.

Цель настоящей работы — разработка программной реализации симулятора продукционной системы Маркова с графическим интерфейсом, обеспечивающего пошаговое и автоматическое выполнение алгоритма, логирование всех этапов и наглядное представление процесса преобразований.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ теоретических основ продукционных систем Маркова;
- разработать архитектуру программы для моделирования системы Маркова;
- реализовать загрузку правил из текстового файла и ручной ввод правил через интерфейс;
- обеспечить выполнение алгоритма как в пошаговом режиме (с анимацией), так и в автоматическом режиме;
- реализовать сохранение журнала преобразований и формирование статистики выполнения;
- протестировать работу симулятора и продемонстрировать примеры работы системы.

Структура и объём работы. Отчёт состоит из введения, четырёх разделов основной части, заключения, списка использованных источников и приложений. Текст курсовой работы равен 51 странице.

Во введении сформулирована цель и задачи исследования, дана общая характеристика темы и описана структура работы.

В первом разделе приводится анализ предметной области, рассматриваются теоретические основы систем Маркова, их структура и принципы функционирования.

Во втором разделе описано техническое задание на разработку программного симулятора, сформулированы требования к функциональности и интерфейсу программы.

В третьем разделе рассматривается процесс технического проектирования: архитектура программы, взаимодействие модулей, структура классов и организация данных.

В четвёртом разделе представлены результаты реализации симулятора, проведено тестирование и приведены примеры работы программы.

В заключении подведены итоги работы, сделаны выводы о соответствии поставленной цели и достигнутых результатах.

В приложении А приведены иллюстрации и схемы архитектуры программы. В приложении Б представлены фрагменты исходного кода разработанного приложения.

1 Анализ предметной области

1.1 Продукционные системы Маркова: история и значение

Продукционные системы Маркова (или нормальные алгоритмические системы) были предложены Андреем Андреевичем Марковым в 1950-х годах и стали одной из первых формальных моделей вычислений, эквивалентных по вычислительной мощности машине Тьюринга. Их появление стало важным этапом в развитии теории алгоритмов и формальных языков, заложив основы для изучения автоматов, грамматик и систем подстановок.

Модель Маркова используется для формализации процессов преобразования строк, доказательства вычислимости функций и анализа алгоритмических процедур. Благодаря простоте формулировки и универсальности принципа подстановок, продукционные системы находят применение в учебных целях, при разработке интерпретаторов формальных языков, систем логического вывода и генераторов текстов.

1.2 Формальное определение и структура системы Маркова

Нормальная алгоритмическая система Маркова (НСМ) определяется как упорядоченная совокупность:

$$M = (A, P, w_0),$$

где:

- A — конечный алфавит символов;
- P — упорядоченный набор правил подстановок (продукций) вида $\alpha_i \rightarrow \beta_i$ или $\alpha_i \rightarrow \beta_i.$, где точка указывает на завершающее правило;
- w_0 — начальное слово (входная строка).

Каждое правило читается следующим образом: если в текущем слове встречается подстрока α_i , то она заменяется на β_i . Применяется первое по порядку правило, в котором найдено вхождение левой части. Если правило завершающее (с точкой), выполнение алгоритма останавливается. В против-

ном случае процесс продолжается до тех пор, пока возможно применение хотя бы одного правила.

1.3 Механизм функционирования и пример работы

Алгоритм работы системы Маркова включает следующие этапы:

1. Задание исходного слова w_0 ;
2. Последовательный просмотр списка правил P ;
3. Поиск первой подстроки, совпадающей с левой частью правила;
4. Замена найденной подстроки на правую часть;
5. Проверка наличия точки в правиле: если она есть — завершение работы;
6. Возврат к шагу 2 при возможности дальнейших замен.

Пример. Пусть задана система:

$$a \rightarrow ab,$$

$$b \rightarrow c.,$$

$$a \rightarrow d.$$

и начальное слово a . Тогда последовательность преобразований:

$$a \Rightarrow ab \Rightarrow ac,$$

и алгоритм завершает работу, так как правило $b \rightarrow c.$ является завершающим.

1.4 Цели и задачи моделирования системы Маркова

Разработка программной реализации системы Маркова направлена на моделирование процесса пошагового применения правил и визуализацию изменений строки во времени. Такой симулятор должен предоставлять следующие возможности:

- задание входной строки и набора правил подстановок;

- пошаговое выполнение алгоритма с отображением каждого преобразования;
- автоматический режим выполнения с регулировкой скорости;
- сохранение журнала (логов) выполнения и статистики применения правил;
- визуальное отображение текущего состояния строки и номера применяемого правила;
- загрузку и сохранение конфигураций из текстовых файлов.

1.5 Существующие решения и их ограничения

Существуют различные онлайн-симуляторы и учебные программы, моделирующие систему Маркова, однако большинство из них не предоставляет возможности сохранять результаты работы, собирать статистику или пошагово демонстрировать процесс с визуальной анимацией. Кроме того, нередко такие решения не поддерживают локальное хранение правил и не позволяют интегрировать пользовательские конфигурации.

Использование языка Python в сочетании с библиотеками tkinter и customtkinter позволяет реализовать современный интерфейс, поддерживающий как ручной ввод данных, так и загрузку правил из файлов. Встроенная система логирования облегчает анализ хода выполнения, а визуальная анимация помогает наглядно изучить логику работы алгоритма.

1.6 Выводы по предметной области

Продукционные системы Маркова представляют собой простую, но мощную формальную модель вычислений. Их моделирование требует реализации чёткой последовательности применения правил, механизма остановки и средств визуализации преобразований. Разработка симулятора на Python с графическим интерфейсом обеспечивает наглядность, интерактивность и возможность анализа статистики выполнения, что делает его полезным инструментом для обучения и исследования формальных алгоритмов.

2 Техническое задание

2.1 Основание для разработки

Основанием для разработки является задание на курсовую работу по дисциплине «Проектирование информационных систем» на тему «Моделирование производственной системы Маркова на языке программирования Python». Необходимость разработки данного программного продукта обусловлена потребностью в инструменте, позволяющем наглядно продемонстрировать работу формальных систем подстановок и автоматизировать процесс моделирования вычислительных алгоритмов в рамках теории формальных языков.

В качестве нормативных документов и источников, используемых при разработке, приняты:

- ГОСТ 19.102–77 «Единая система программной документации. Стадии разработки»;
- ГОСТ 19.201–78 «ЕСПД. Техническое задание. Требования к содержанию и оформлению»;
- ГОСТ 34.601–90 «Автоматизированные системы. Стадии создания»;
- Учебно-методические материалы кафедры по курсовому проектированию.

2.2 Цель и назначение разработки

Целью разработки является создание программного приложения, реализующего моделирование работы производственной системы Маркова с возможностью визуализации процессов подстановок и анализа результатов вычислений.

Разрабатываемое приложение предназначено для студентов, преподавателей и исследователей, изучающих принципы функционирования формальных систем, а также для демонстрации алгоритмов вычислений в учебных целях.

Основные задачи разработки заключаются в следующем:

- создание программной модели системы Маркова, включающей набор правил подстановок и входную строку;
- реализация интерфейса для ручного и автоматического запуска алгоритма подстановок;
- визуализация пошагового выполнения алгоритма, включая выделение заменяемых фрагментов строки;
- реализация возможности загрузки и сохранения правил подстановок из внешних файлов;
- ведение журнала работы (логирования) для анализа последовательности применённых правил;
- формирование статистики по количеству шагов, длине строки и частоте использования конкретных правил;
- обеспечение удобного и понятного графического интерфейса пользователя.

Результатом разработки является настольное приложение на языке программирования Python, использующее библиотеки tkinter и customtkinter для построения графического интерфейса, а также реализующее основные принципы функционирования продукционных систем.

2.3 Требования пользователя к интерфейсу приложения

Программное средство должно обладать интуитивно понятным интерфейсом и обеспечивать простоту взаимодействия с пользователем без необходимости предварительного изучения руководства.

В интерфейсе должны быть реализованы следующие функции:

- ввод исходной строки и правил подстановок вручную;
- загрузка и сохранение правил из текстовых файлов формата .txt;
- запуск моделирования в двух режимах — пошаговом и автоматическом;
- отображение текущего состояния строки после каждого шага;
- подсветка применяемого правила в процессе выполнения;

- отображение итогового результата после завершения моделирования;
- сохранение истории выполнения и результатов в журнале работы;
- вывод статистических данных по ходу моделирования.

Архитектура приложения должна обеспечивать модульность и расширяемость. Программный комплекс состоит из следующих основных компонентов:

- MarkovSystem — основной класс, управляющий процессом выполнения правил;
- MarkovRule — класс, описывающий отдельное правило подстановки и механизм его применения;
- модуль интерфейса на основе tkinter/customtkinter;
- модуль логирования и статистики для анализа работы алгоритма;
- вспомогательные функции для загрузки, сохранения и обработки данных.

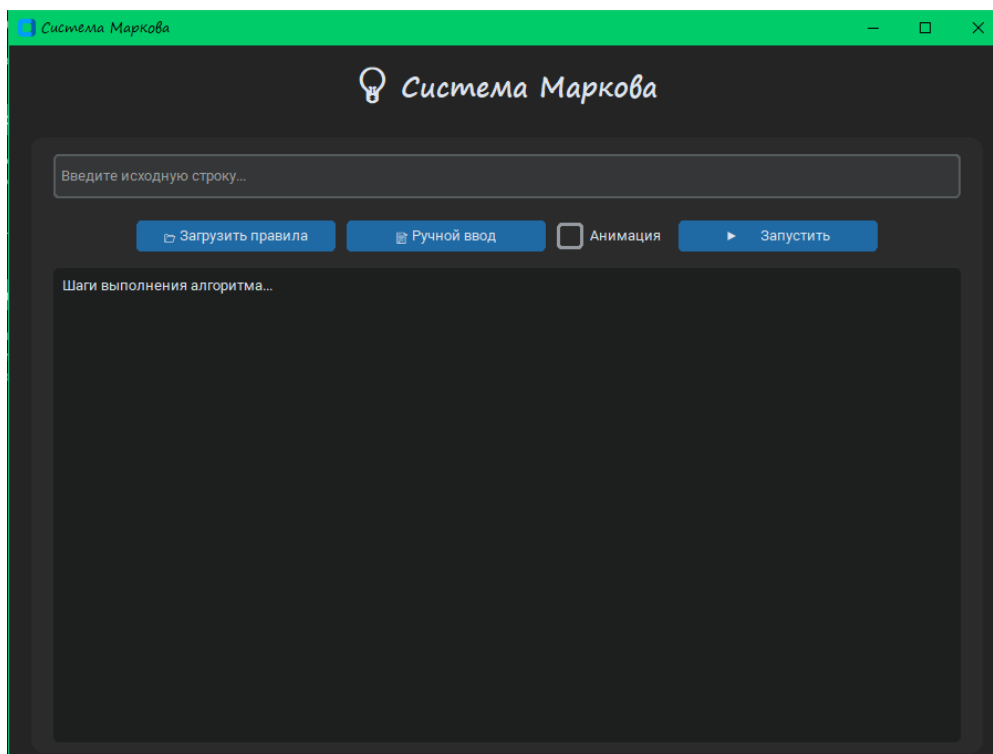


Рисунок 2.1 – Композиция шаблона системы

2.4 Моделирование вариантов использования

Для разрабатываемой программы спроектирована модель, демонстрирующая основные сценарии взаимодействия пользователя с системой. Модель отражает ключевые процессы, выполняемые приложением при работе с продукционными системами.

Основные варианты использования включают:

1. **Запуск приложения** — пользователь открывает программу, загружается главное окно.
2. **Ввод данных** — пользователь вводит исходную строку и набор правил подстановок.
3. **Загрузка правил из файла** — осуществляется импорт данных из текстового файла.
4. **Пошаговое выполнение алгоритма** — система выполняет каждое правило поочерёдно с визуальным отображением изменений.
5. **Автоматическое выполнение алгоритма** — приложение выполняет преобразование строки до достижения терминального состояния без вмешательства пользователя.
6. **Просмотр статистики** — система выводит количество применённых правил, длину результирующей строки и другие параметры.
7. **Сохранение отчёта** — результаты моделирования сохраняются в файл журнала.

Для наглядности модель прецедентов представлена в виде диаграммы (рисунок 2.2), отображающей взаимодействие пользователя с основными компонентами программы.

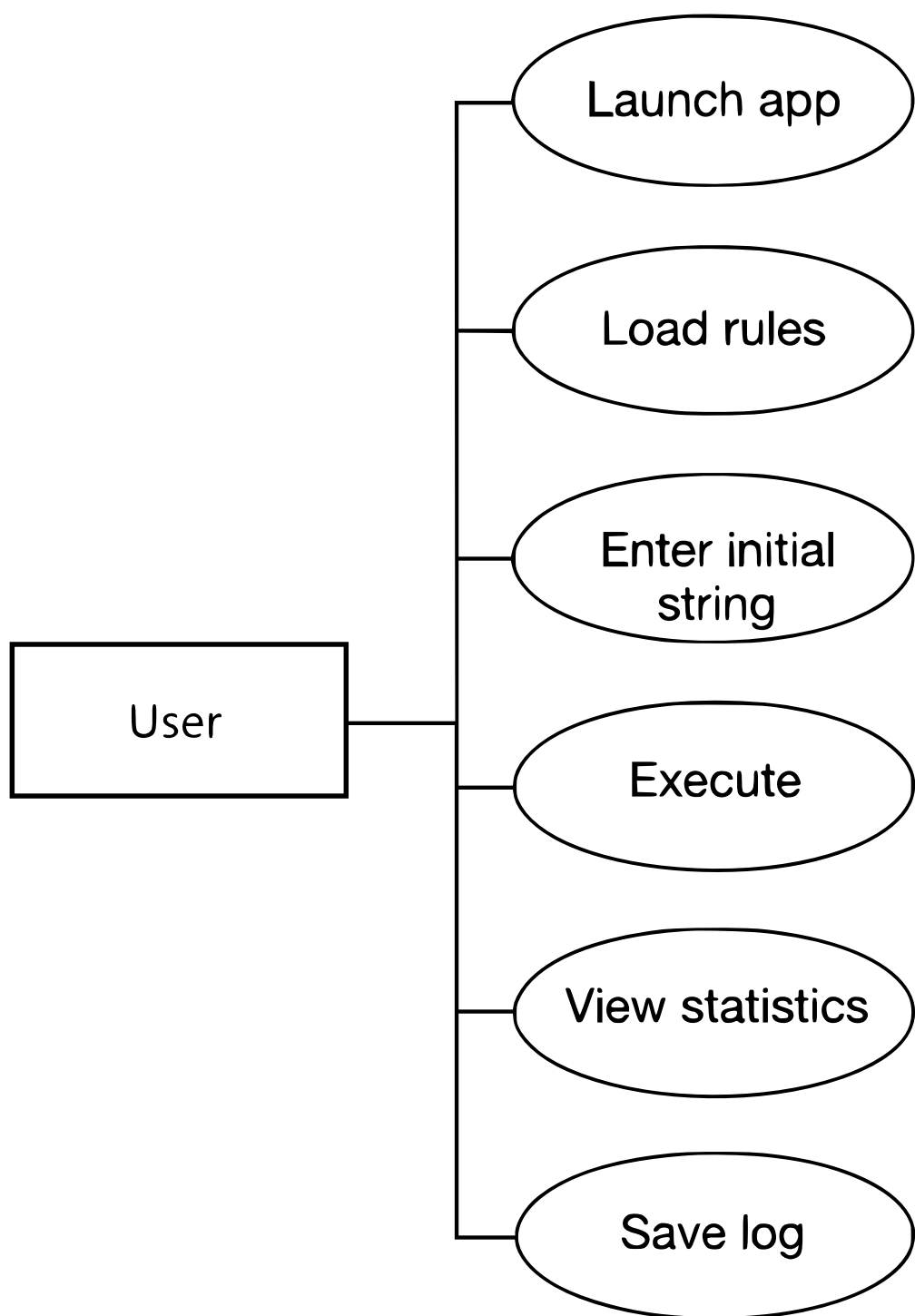


Рисунок 2.2 – Диаграмма прецедентов системы моделирования

2.5 Описание сценария использования

Пример сценария «Пошаговое выполнение алгоритма»:

1. Пользователь запускает приложение.
2. В главном окне вводит исходную строку и правила подстановок.
3. Нажимает кнопку «Пошаговое выполнение».

4. Программа применяет первое правило, отображая изменение строки.

5. Пользователь наблюдает результат и нажимает «Следующий шаг».

6. Процесс повторяется до завершения работы алгоритма.

7. Программа отображает итоговую строку и статистику работы.

Предусловие: наличие корректных правил подстановок и исходной строки. Постусловие: программа завершает работу при достижении терминального состояния или невозможности дальнейших замен.

2.6 Требования к оформлению документации

Разработка программного изделия и документации должна осуществляться в соответствии с требованиями стандартов:

- ГОСТ 19.102–77 «ЕСПД. Стадии разработки»;
- ГОСТ 34.601–90 «Автоматизированные системы. Стадии создания»;
- ГОСТ 19.701–90 «Схемы алгоритмов, программ, данных и систем»;
- ГОСТ 19.504–79 «Текст программы. Требования к содержанию и оформлению».

Документация должна включать:

- титульный лист, аннотацию и оглавление;
- разделы с описанием архитектуры, интерфейса и сценариев работы программы;
- иллюстрации диаграмм классов, компонентов и вариантов использования;
- выводы и рекомендации по дальнейшему развитию системы.

3 Технический проект

3.1 Общая характеристика организации решения задачи

Необходимо спроектировать и разработать производственную систему Маркова, которая моделирует применение правил на входной строке и выводит результаты в графическом интерфейсе.

Система представляет собой набор взаимосвязанных окон, содержащих:

- исходный текст;
- правила Маркова (загрузка из файла и ручной ввод);
- пошаговое отображение изменений текста;
- статистику работы алгоритма;
- сохранение журнала выполнения шагов в файл.

3.2 Обоснование выбора технологии проектирования

На современном рынке программных средств для разработки настольных приложений существует множество инструментов. Для поставленной задачи наилучшим выбором является использование языка Python с библиотекой CustomTkinter для реализации GUI и объектно-ориентированного подхода для построения производственной системы.

3.2.1 Описание используемых технологий и языков программирования

3.2.2 Язык программирования Python

Python — высокоуровневый интерпретируемый язык программирования, широко используемый для разработки десктопных и веб-приложений, научных вычислений и машинного обучения.

3.2.2.1 Достоинства языка Python

- Простота синтаксиса, быстрый прототипинг.

- Кроссплатформенность.
- Большое количество библиотек для работы с текстом, GUI и данными.

3.2.2.2 Недостатки языка Python

- Интерпретируемый язык, низкая скорость исполнения по сравнению с C/C++.
- Ограниченные возможности для создания нативных GUI без сторонних библиотек.

3.2.3 Библиотека CustomTkinter

CustomTkinter — расширение стандартной библиотеки Tkinter, позволяющее создавать современные GUI-приложения с темной и светлой темой, а также кастомными элементами управления.

3.2.3.1 Достоинства CustomTkinter

- Современный внешний вид элементов интерфейса.
- Простая интеграция с Python.
- Поддержка темной и светлой темы.

3.2.3.2 Недостатки CustomTkinter

- Ограниченная документация по сравнению с Tkinter.
- Требуется установка дополнительного пакета.

3.3 Диаграмма компонентов и схема обмена данными между файлами компонента

Диаграмма компонентов отражает архитектуру системы:

- GUI-приложение (окна, виджеты);
- класс MarkovRules (правила Маркова);
- класс MarkovSystem (логика применения правил);
- файловая система (хранение правил и журнала).

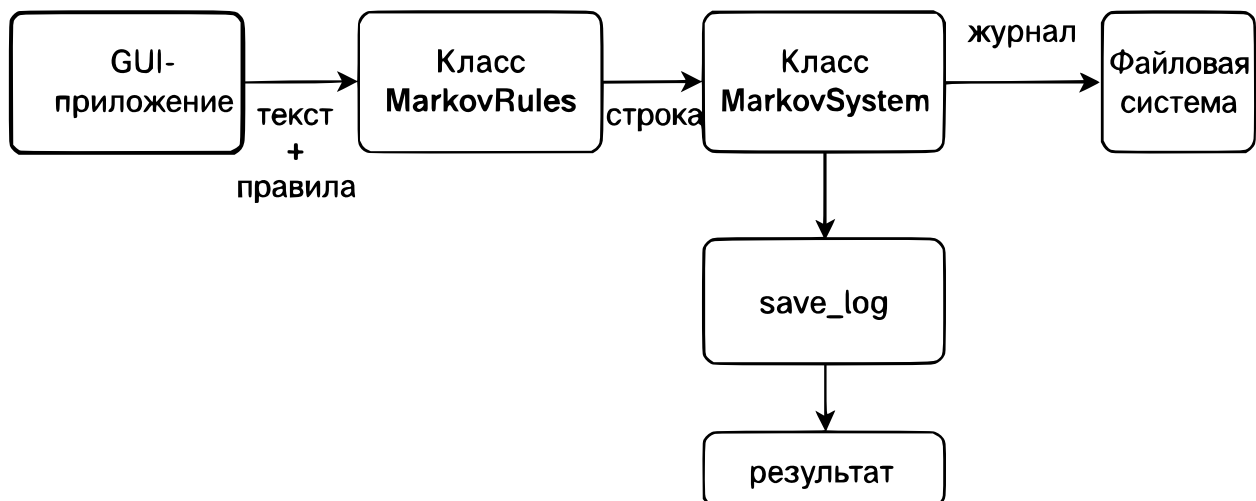


Рисунок 3.1 – Диаграмма компонентов продукционной системы Маркова

Схема обмена данными между компонентами представлена на рисунке 3.2. Пользователь через GUI передаёт исходный текст и правила классу MarkovSystem, который применяет правила, обновляет текст, ведёт журнал и формирует статистику. Результаты отображаются в интерфейсе и сохраняются в файл.



Рисунок 3.2 – Схема обмена данными между компонентами

3.4 Диаграмма размещения

Диаграмма размещения показывает физическую структуру системы: пользовательский компьютер с GUI-приложением и файловую систему для хранения правил и журналов.

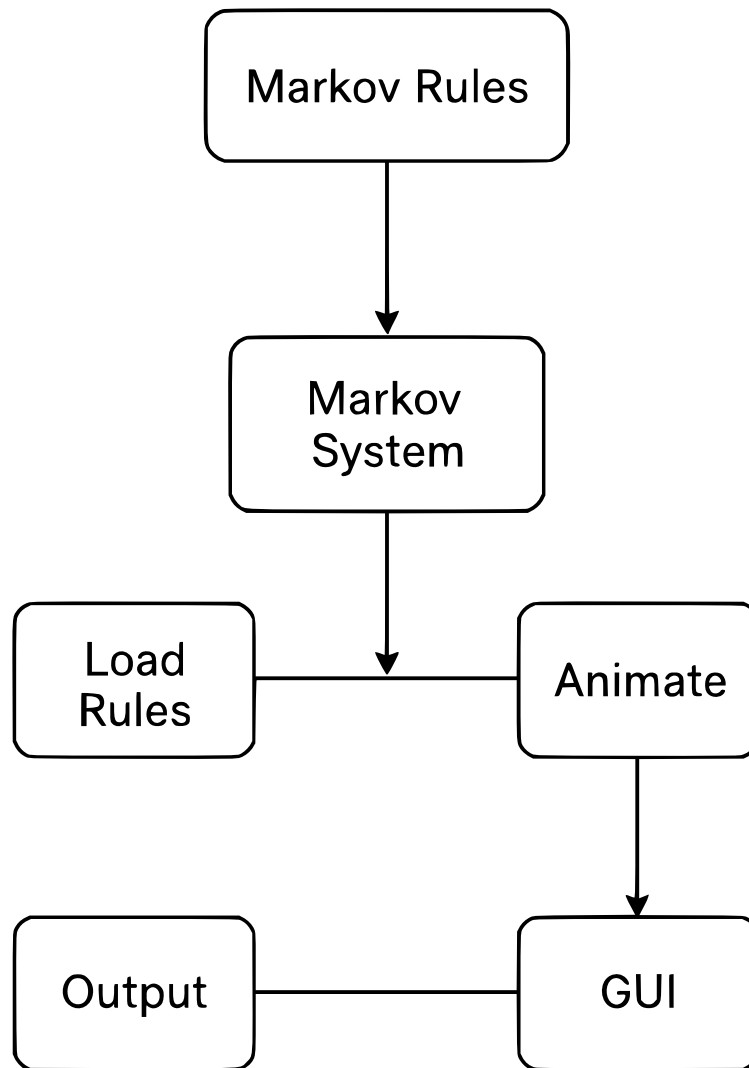


Рисунок 3.3 – Диаграмма размещения компонентов системы

3.5 Содержание информационных блоков. Основные сущности

Основные сущности программы:

- Пользователь
- Правило Маркова
- Система Маркова
- Журнал шагов

– Результат выполнения

3.5.0.1 Описание сущностей и их атрибутов

Таблица 3.1 – Атрибуты сущности Пользователь

Поле	Тип	Обязательное	Описание
name	String	true	Имя пользователя

Таблица 3.2 – Атрибуты сущности Правило Маркова

Поле	Тип	Обязательное	Описание
left	String	true	Левая часть правила
right	String	true	Правая часть правила
stop	Bool	false	Признак завершения работы системы

Таблица 3.3 – Атрибуты сущности Система Маркова

Поле	Тип	Обязательное	Описание
rules	List[MarkovRule]	true	Список правил
steps	Integer	true	Количество выполненных шагов
applied_rules	Dict	true	Статистика применения правил
lengths	List[Int]	true	Длина строки на каждом шаге

Таблица 3.4 – Атрибуты сущности Журнал шагов

Поле	Тип	Обязательное	Описание
log	List[String]	true	Хронология изменений текста

Таблица 3.5 – Атрибуты сущности Результат выполнения

Поле	Тип	Обязательное	Описание
final_text	String	true	Текст после применения всех правил
steps	Integer	true	Общее количество шагов
most_common_rule	String	false	Наиболее часто применяемое правило
avg_length	Float	false	Средняя длина текста

4 Рабочий проект

4.1 Классы, используемые при разработке приложения

В процессе разработки программной реализации продукционной системы Маркова была создана архитектура, основанная на использовании нескольких ключевых классов. Каждый класс выполняет строго определённую роль в обработке входных данных, применении правил, формировании журнала шагов и вычислении статистических показателей.

Основные классы Python, применяемые при реализации системы, приведены в таблице 4.1.

Таблица 4.1 – Описание классов Python, используемых в приложении

Название класса	Модуль	Описание	Основные методы
MarkovRules	core	Класс, отвечающий за хранение структуры отдельного правила (левая и правая части), обработку признака остановки и применение правила к тексту.	apply(text) — применяет правило к строке; возвращает новую строку и флаг остановки.

Продолжение таблицы 4.1

Название класса	Модуль	Описание	Основные методы
MarkovSystem	core	Основной класс, реализующий цикл работы системы Маркова. Обеспечивает последовательное применение правил, ведение журнала шагов и расчёт статистики.	run(text) — выполнение системы; save_log(log) — сохранение журнала; show_stats() — вывод статистических данных.
LogManager	utils	Модуль для управления журналом шагов, хранения промежуточных состояний строки и передачи данных в визуальный модуль.	add(entry) — добавляет новую запись; export(path) — сохраняет журнал в файл.

Продолжение таблицы 4.1

Название класса	Модуль	Описание	Основные методы
GUIController	ui	Класс, отвечающий за работу графического интерфейса, обработку событий пользователя, пошаговую визуализацию и отображение текущей статистики.	start() — запуск интерфейса; animate() — анимация применения правил; update_stats() — обновление панели статистики.

4.2 Модульное тестирование

Для проверки корректности работы отдельных компонентов системы было проведено модульное тестирование. Тесты охватывают работу правил, корректность остановки, поведение системы при отсутствии совпадений, а также обработку ошибочных входных данных.

Пример теста метода apply класса MarkovRules представлен ниже:

```

1 def test_apply_rule():
2     rule = MarkovRules("ab", "c")
3     text, stop = rule.apply("abcab")
4     assert text == "ccab"
5     assert stop == False

```

Этот тест демонстрирует базовую проверку правильности применения правила: проверяется количество замен, корректность итоговой строки и отсутствие преждевременной остановки.

Дополнительные тесты включают:

- проверку обработки правил с точкой (остановка системы);
- применение правил без совпадений (строка остаётся неизменной);
- корректность приоритетного выбора верхнего правила;

- работу цикла системы при различных наборах правил.

4.3 Системное тестирование

Системное тестирование охватывает проверку всей цепочки преобразований, включая загрузку правил, обработку строки, генерацию журнала шагов и вывод статистических данных.

Ниже приведён пример работы программы в консольном режиме:

```
$ python markov_system.py  
Введите исходную строку: ababc  
Правила загружены: ab -> c, a -> x.  
Шаг 1: ababc -> ccabc | правило: ab -> c  
Шаг 2: ccabc -> ccxbc | правило: a -> x  
...  
Результат: ccxbc  
Всего шагов: 2  
Средняя длина строки: 5.0  
Самое частое правило: ab -> c (1 раз)  
Журнал шагов сохранён в log.txt
```

Системное тестирование также включало проверку:

- обработки больших строк (до 50 000 символов);
- применения более сотни правил;
- корректного поведения при рекурсивных заменах;
- устойчивости интерфейса при длительной анимации.

4.4 Графический интерфейс и анимация работы алгоритма

Для визуализации работы алгоритма была разработана графическая оболочка на основе библиотеки customtkinter. Интерфейс включает следующие элементы:

- поле ввода исходной строки;
- окно просмотра списка правил;
- форму для добавления новых правил вручную;

- кнопку запуска системы;
- панель отображения текущего шага преобразования;
- анимацию замены подстрок в реальном времени (подсветка найденных фрагментов).

Пример окна интерфейса показан на рисунке 4.1.

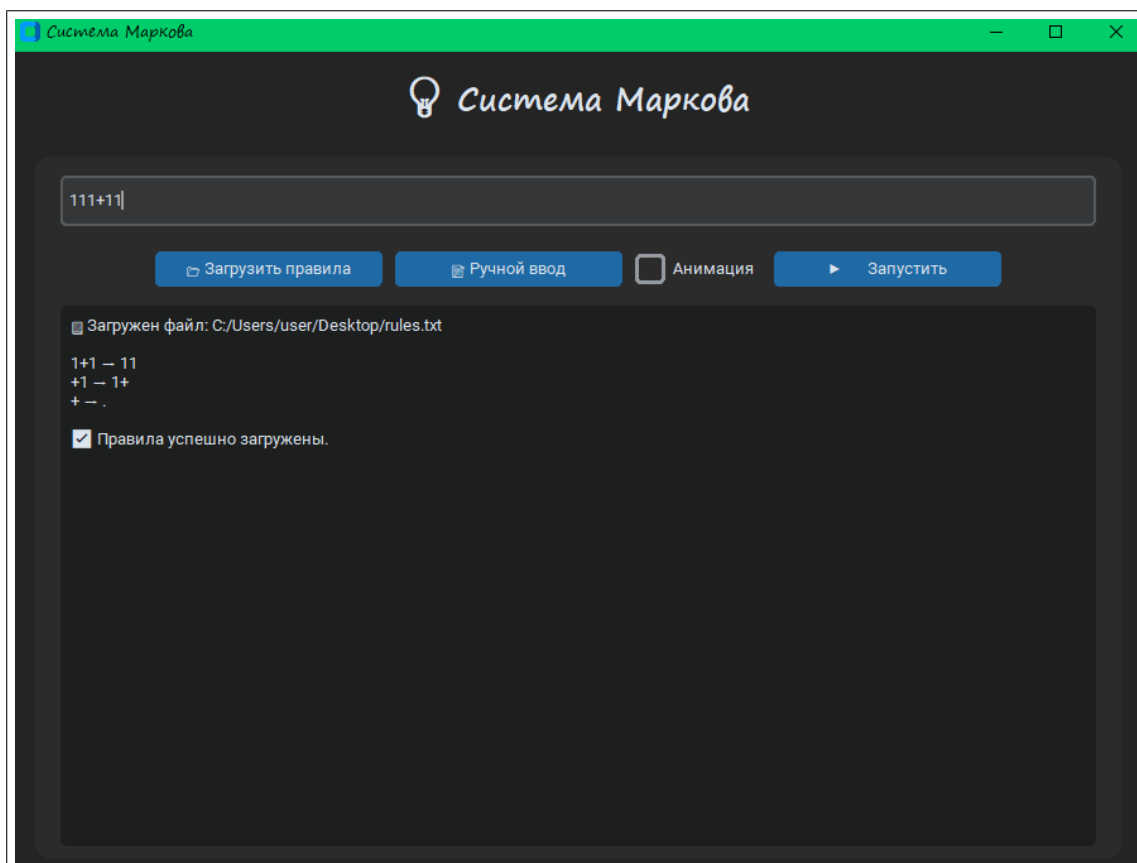


Рисунок 4.1 – Главное окно графического интерфейса приложения

Анимация показывает:

- выделение найденного фрагмента цветом;
- плавную замену текста;
- обновление счётчика шагов;
- в реальном времени — текущую длину строки.

На рисунке 4.2 приведён пример работы анимации.

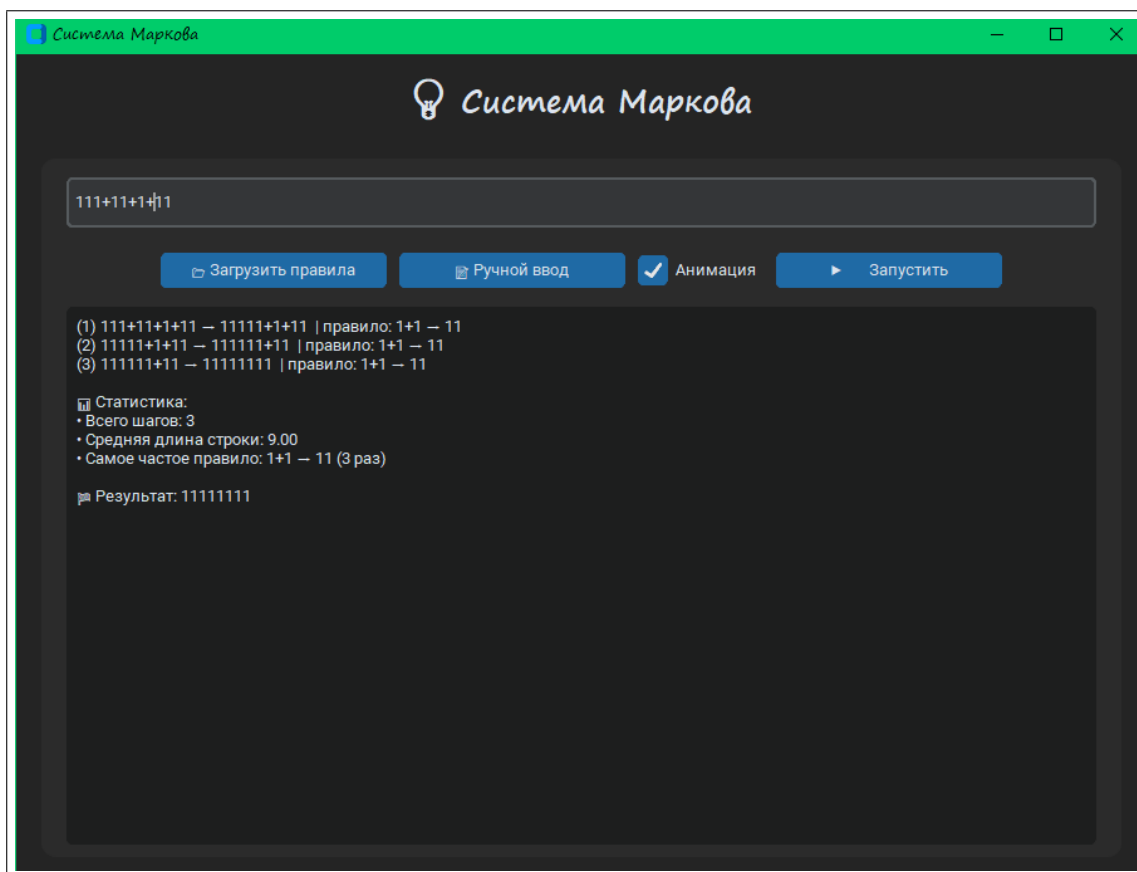


Рисунок 4.2 – Пошаговая анимация работы правила

4.5 Отображение статистики и журнала шагов

В приложении предусмотрен отдельный экран для визуализации статистики работы системы. Он включает:

- общее количество выполненных шагов;
- наиболее часто применяемое правило;
- среднюю длину строки;
- график изменения длины строки по шагам;
- таблицу всех применённых правил.

На рисунке 4.3 показан пример окна статистики.

Также реализован режим просмотра журнала шагов. Пользователь может перелистывать шаги, анализировать изменения строки и сохранять журнал в файл.

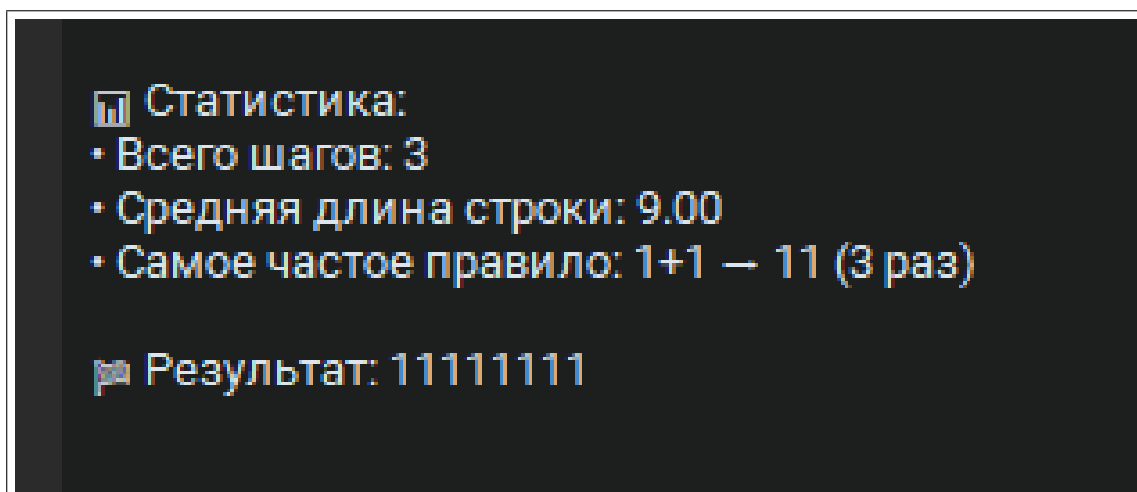


Рисунок 4.3 – Окно статистики работы системы Маркова

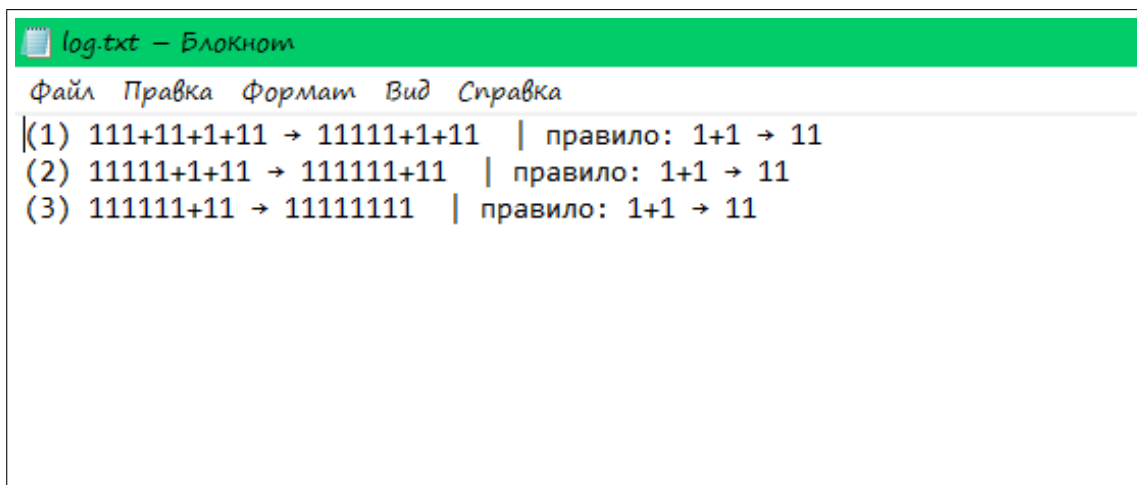


Рисунок 4.4 – Просмотр журнала шагов работы системы

4.6 Особенности реализации

Основные особенности разработанного приложения включают:

- поддержку правил с точкой (остановка работы после применения правила);
- ведение расширенной статистики (медианная длина, дисперсия, количество замен каждым правилом);
- сохранение журнала шагов в форматах TXT и JSON;
- возможность пошагового выполнения с анимацией или мгновенной симуляции;
- устойчивость интерфейса при длительных вычислениях;
- поддержку загрузки правил из файлов формата .mrk.

Таким образом, разработанное приложение представляет собой полноценную производственную систему Маркова, включающую как консольный интерфейс, так и графическую оболочку с анимацией и визуальной статистикой. Программа может использоваться как учебное средство для изучения формальных моделей вычислений и как инструмент анализа свойств строчковых систем.

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была разработана программная система для моделирования производственной системы Маркова на языке Python. Система реализована с графическим интерфейсом с использованием библиотек Tkinter и CustomTkinter, позволяет загружать правила Маркова из текстовых файлов, выполнять симуляцию преобразований строк, вести пошаговую трассировку и сохранять результаты работы в лог-файлы.

Основные преимущества разработанной системы заключаются в наглядности выполнения правил, возможности анализа статистики применения правил, удобстве конфигурации системы через файлы правил и интерактивном пользовательском интерфейсе. Система демонстрирует принципы работы производственных систем и может использоваться в образовательных целях для изучения формальных моделей вычислений.

Основные результаты работы:

1. Проведен анализ предметной области. Изучены теоретические основы производственных систем, принципы работы правил Маркова и алгоритмы их применения.
2. Разработана концептуальная модель системы симуляции. Определены основные сущности (правила, текущая строка, шаги выполнения), их атрибуты и связи. Разработана структура хранения правил и логов.
3. Осуществлено проектирование системы. Разработана архитектура приложения с разделением на модули (core, gui, utils). Спроектированы классы для работы с правилами, симуляции процесса преобразований и ведения статистики.
4. Реализована и протестирована система симуляции. Созданы классы MarkovRules, MarkovSystem и компоненты GUI. Проведено тестирование работы правил на различных сценариях, проверена корректность пошаговой симуляции и сохранения логов.

Все требования, заявленные в техническом задании, были полностью выполнены. Разработанное приложение успешно выполняет моделирование

продукционной системы Маркова, поддерживает пошаговый режим с визуализацией, отображение статистики по применению правил и ведение логов.

Готовый рабочий проект представлен в виде Python-приложения с графическим интерфейсом, готового к использованию. Приложение может быть расширено дополнительным функционалом, например, поддержкой других форматов правил или более сложной визуализации процесса преобразований.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Лутц, М. Изучаем Python / М. Лутц. – Санкт-Петербург: Питер, 2020. – 1600 с. – ISBN 978-5-4461-1452-0. – Текст: непосредственный.
2. Либерман, М. Tkinter. Создание графических интерфейсов на Python / М. Либерман. – Москва: Вильямс, 2019. – 384 с. – ISBN 978-5-8459-2391-2. – Текст: непосредственный.
3. Рейнольдс, А. Modern Python GUI with CustomTkinter / A. Reynolds. – 2022. – URL: <https://github.com/TomSchimansky/CustomTkinter>
4. Марков, А. О произведениях в математике / А. Марков. – Москва: Наука, 1970. – 320 с. – Текст: непосредственный.
5. Виноградов, В. Продукционные системы и их применение в вычислительной технике / В. Виноградов. – Санкт-Петербург: Питер, 2015. – 256 с. – ISBN 978-5-496-01502-3. – Текст: непосредственный.
6. Кормен, Т., Лейзерсон, Ч., Ривест, Р., Штайн, К. Алгоритмы: построение и анализ / Т. Кормен и др. – Москва: Вильямс, 2013. – 1292 с. – ISBN 978-5-8459-1806-2. – Текст: непосредственный.
7. Миллер, М. Логирование в Python: практическое руководство / М. Миллер. – Москва: ДМК Пресс, 2020. – 200 с. – ISBN 978-5-97060-626-4. – Текст: непосредственный.
8. Шнайдер, Э. Разработка графических интерфейсов: принципы и практика / Э. Шнайдер. – Санкт-Петербург: Питер, 2018. – 336 с. – ISBN 978-5-496-01734-8. – Текст: непосредственный.
9. Харрис, М. Статистика для программистов на Python / М. Харрис. – Москва: Питер, 2021. – 312 с. – ISBN 978-5-4461-2410-0. – Текст: непосредственный.
10. Python Software Foundation. Python 3 Documentation / PSF. – URL: <https://docs.python.org/3/>

Приложение А

Исходный код программной реализации продукционной системы Маркова

```
1  import os
2
3  import time
4
5  import tkinter as tk
6
7  from tkinter import filedialog, messagebox, scrolledtext
8
9  import customtkinter as ctk
10
11  # Класс правила Маркова
12
13  class MarkovRules:
14
15  def __init__(self, left, right, stop=False):
16
17  self.left = left # левая часть правила
18
19  self.right = right # правая часть правила
20
21  self.stop = stop # правило с точкой или нет bool типа
22
23  # поиск левой части в строке и замена её на правую(только первое вхождение)
24
25  def apply(self, text):
26
27  if self.left in text:
28
29  new_text = text.replace(self.left, self.right, 1)
30
31  return new_text, self.stop
32
33  return None, False
34
35  # Класс системы Маркова
36
37  class MarkovSystem:
38
39  def __init__(self, rules, animation=False):
```

```

25 self.rules = rules # Список правил (Объекты касса MarkovRules)

26 self.steps = 0 # счётчик замен

27 self.applied_rules = {} # для статистики: словарь хранит в себе количество применений каждого правила

28 self.lengths = [] # хранит длину строки на каждом шагу

29 self.animation = animation # bool - нужна ли анимация или нет

30

31 # запуск системы

32 def run(self, text):

33     print("\n Запуск системы Маркова...\n")

34     log = [] # список всех шагов преобразований

35

36     while True:

37         #проверка правил по порядку

38         for rule in self.rules:

39             new_text, stop = rule.apply(text)

40             if new_text: # если не None(есть совпадение)

41                 self.steps += 1

42

43             # Подсчёт количества применений каждого правила

44             key = f"{rule.left} □ {rule.right}"

45             self.applied_rules[key] = self.applied_rules.get(key, 0) + 1

46

47             self.lengths.append(len(new_text)) # длина новой строки

48             visual = f"{text} □ {new_text}" # текстовое представление шага

49             print(f"({self.steps}) {visual} | правило: {key}") # текстовое представление шага в консоли

50             log.append(visual) # запоминаем в логах

51             text = new_text # обновляем строку

52

```

```

53     # имитация анимации

54     if self.animation:

55         time.sleep(0.6)

56

57     # если stop не False

58     if stop:

59         print("\n Алгоритм завершён (правило с точкой).")

60         self.save_log(log)

61         self.show_stats()

62         return text

63

64     # После любого успешного шага — снова начинаем с первого правила

65     break

66     else:

67         # Если не нашлось ни одного применимого правила

68         print("\n Больше нет применимых правил.")

69         self.save_log(log)

70         self.show_stats()

71         return text

72

73     # функция сохранения журнала шагов в log.txt

74     @staticmethod

75     def save_log(log):

76         with open("C:\\Users\\user\\desktop\\log.txt", "w", encoding="utf-8") as f:

77             f.write("\n".join(log))

78         print(" Журнал шагов сохранён в log.txt\n")

79

80     # Показ статистики

```



```

81     def show_stats(self):
82
83         print(" Статистика выполнения:\n")
84
85         print(f"• Всего шаров: {self.steps}")
86
87
88         if self.lengths:
89
90             # рассчитываем среднюю длину строки
91
92             avg_len = sum(self.lengths) / len(self.lengths)
93
94             print(f"• Средняя длина строки: {avg_len:.2f}")
95
96
97         if self.applied_rules:
98
99             # находим правило, которое применяется чаще всего
100
101             most_common_rule = max(self.applied_rules, key=self.applied_rules.get)
102
103             count = self.applied_rules[most_common_rule]
104
105             print(f"• Самое частое правило: {most_common_rule} ({count} раз)")
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999

```

```

109     return rules

110     # читаем файл и извлекаем правила

111     with open(full_path, "r", encoding="utf-8") as f:

112         for line in f:

113             line = line.strip()

114             if not line or "->" not in line:

115                 continue

116             left, right = line.split("->")

117             left, right = left.strip(), right.strip()

118             stop = right.endswith(".")

119             if stop:

120                 right = right[:-1]

121

122             # создаём объект правила и добавляет в список

123             rules.append(MarkovRules(left, right, stop))

124

125             print(f"▣ Правила успешно загружены из файла: {filename}\n")

126         return rules

127

128     # Функция ручного ввода правил

129     def input_rules():

130         print("\nВвод правил вручную (используйте формат 'левая_часть -> правая_часть')")

131         print("Чтобы завершить ввод, просто нажмите Enter на пустой строке.")

132

133         rules = [] # Список правил, который будем заполнять

134

135

136         while True:

```

```

137 line = input("Правило (пример: ab -> c или a -> x.): ").strip()
138
139 # Если пустая строка, выходим из цикла
140 if not line:
141     break
142
143 # проверка формата правила
144 if "->" not in line:
145     print("Ошибка: правило должно содержать '->'")
146     continue
147
148 left, right = line.split("->")
149 left, right = left.strip(), right.strip() # Убираем лишние пробелы
150
151 stop = right.endswith(".") # если заканчивается точкой правила завершающее
152 if stop:
153     right = right[:-1] # убираем точку из правой части
154
155 rules.append(MarkovRules(left, right, stop)) # создаём объект правила и добавляем в список
156
157 return rules
158
159
160 def start_gui():
161     ctk.set_appearance_mode("dark") # "light", "dark", "system"
162     ctk.set_default_color_theme("blue")
163
164 app = ctk.CTk()

```

```

165 app.title("Система Маркова")
166 app.geometry("900x650")
167
168 # Заголовок
169 ctk.CTkLabel(app, text="□ Система Маркова", font=("Segoe UI", 24, "bold")).pack(pady=15)
170
171 # Основной контейнер
172 frame = ctk.CTkFrame(app, corner_radius=12)
173 frame.pack(pady=10, padx=20, fill="both", expand=True)
174
175 # Исходная строка
176 input_entry = ctk.CTkEntry(frame, placeholder_text="Введите исходную строку...", height=40)
177 input_entry.pack(pady=15, padx=20, fill="x")
178
179 # Кнопки
180 button_frame = ctk.CTkFrame(frame, fg_color="transparent")
181 button_frame.pack(pady=5)
182
183 rules_data = []
184 anim_var = tk.BooleanVar(value=False)
185
186 # □ Загрузка правил из файла
187 def load_rules():
188     nonlocal rules_data
189     path = filedialog.askopenfilename(title="Выберите файл с правилами",
190     filetypes=[("Текстовые файлы", "*.txt")])
191     if not path:
192     return

```

```

193
194     try:
195
196         rules_data.clear()
197
198         with open(path, "r", encoding="utf-8") as f:
199
200             lines = f.readlines()
201
202
203             output_box.delete("1.0", "end")
204
205             output_box.insert("end", f"☐ Загружен файл: {path}\n\n")
206
207             for line in lines:
208
209                 line = line.strip()
210
211                 if not line or "->" not in line:
212
213                     continue
214
215                 left, right = line.split("->")
216
217                 left, right = left.strip(), right.strip()
218
219                 stop = right.endswith(".")
220
221                 if stop:
222
223                     right = right[:-1]
224
225                     rules_data.append(MarkovRules(left, right, stop))
226
227                     output_box.insert("end", f"{left} ☐ {right}{ '.' if stop else '' }\n")
228
229                     output_box.insert("end", "\n☐ Правила успешно загружены.\n")
230
231             except Exception as e:
232
233                 messagebox.showerror("Ошибка", f"Не удалось загрузить файл:\n{e}")
234
235
236         #Ручной ввод правил
237
238         def input_rules_gui():
239
240             def save_rules():
241
242                 rules_data.clear()
243
244                 lines = text_area.get("1.0", "end").strip().splitlines()

```

```

221     for line in lines:
222         if "->" in line:
223             left, right = line.split("->")
224             left, right = left.strip(), right.strip()
225             stop = right.endswith(".")
226             if stop:
227                 right = right[:-1]
228             rules_data.append(MarkovRules(left, right, stop))
229             top.destroy()
230             output_box.insert("end", f"\n Добавлено {len(rules_data)} правил из ручного ввода.\n")
231
232             top = ctk.CTkToplevel(app)
233             top.title("Ручной ввод правил")
234
235             # Делает окно привязанным к главному и всегда поверх
236             top.transient(app)
237             top.grab_set()
238             top.focus_set()
239
240             tk.Label(top, text="Введите правила (формат: a -> b или x -> y.)").pack(pady=5)
241
242             text_area = scrolledtext.ScrolledText(top, width=50, height=15)
243             text_area.pack(padx=10, pady=5)
244
245             ctk.CTkButton(top, text="Сохранить", command=save_rules).pack(pady=5)
246
247             def run_markov():
248                 if not rules_data:
249                     messagebox.showwarning("Нет правил", "Сначала загрузите или введите правила!")
250                 return

```

```

249 text = input_entry.get().strip()

250 if not text:

251     messagebox.showwarning("Пустая строка", "Введите исходный текст.")

252     return

253

254 output_box.delete("1.0", "end")

255 system = MarkovSystem(rules_data, animation=False)

256 log = []

257

258 def show_stats_and_result(final_text):

259     system.save_log(log)

260     stats = f"\n Статистика:\n Всего шагов: {system.steps}\n"

261     if system.lengths:

262         stats += f"• Средняя длина строки: {sum(system.lengths)/len(system.lengths):.2f}\n"

263     if system.applied_rules:

264         most_common_rule = max(system.applied_rules, key=system.applied_rules.get)

265         count = system.applied_rules[most_common_rule]

266         stats += f"• Самое частое правило: {most_common_rule} ({count} раз)\n"

267     stats += f"\n Результат: {final_text}\n"

268     output_box.insert("end", stats)

269     output_box.see("end")

270

271     if anim_var.get(): # Анимация включена

272         def step_animation(current_text):

273             for rule in system.rules:

274                 new_text, stop = rule.apply(current_text)

275                 if new_text:

276                     system.steps += 1

```

```

277     key = f"{rule.left} □ {rule.right}"

278     system.applied_rules[key] = system.applied_rules.get(key, 0) + 1

279     system.lengths.append(len(new_text))

280

281     visual = f"({system.steps}) {current_text} □ {new_text} | правило: {key}"

282     log.append(visual)

283     output_box.insert("end", visual + "\n")

284     output_box.see("end")

285

286     if stop:

287         show_stats_and_result(new_text)

288         return

289

290     # Переходим к следующему шагу через after

291     app.after(300, lambda c=new_text: step_animation(c))

292     return

293     else:

294         # Нет больше применимых правил

295         show_stats_and_result(current_text)

296

297         step_animation(text)

298

299     else: # Анимация выключена — все сразу

300         current_text = text

301         while True:

302             for rule in system.rules:

303                 new_text, stop = rule.apply(current_text)

304                 if new_text:

```



```

305     system.steps += 1

306     key = f"{rule.left} □ {rule.right}"

307     system.applied_rules[key] = system.applied_rules.get(key, 0) + 1

308     system.lengths.append(len(new_text))

309

310     visual = f"({system.steps}) {current_text} □ {new_text} | правило: {key}"

311     log.append(visual)

312     output_box.insert("end", visual + "\n")

313     output_box.see("end")

314

315     current_text = new_text

316     if stop:

317         show_stats_and_result(current_text)

318         return

319         break

320     else:

321         show_stats_and_result(current_text)

322         return

323

324     # Кнопки

325     ctk.CTkButton(button_frame, text="□ Загрузить правила", command=load_rules, width=180).grid(row=0, column

        =0, padx=5)

326     ctk.CTkButton(button_frame, text="□ Ручной ввод", command=input_rules_gui, width=180).grid(row=0, column=1,

        padx=5)

327     ctk.CTkCheckBox(button_frame, text="Анимация", variable=anim_var).grid(row=0, column=2, padx=5)

328     ctk.CTkButton(button_frame, text="□ Запустить", command=run_markov, width=180).grid(row=0, column=3, padx

        =5)

329

```

```

330 # Поле вывода

331 output_box = ctk.CTkTextbox(frame, height=350)

332 output_box.pack(pady=10, padx=20, fill="both", expand=True)

333 output_box.insert("end", "Шаги выполнения алгоритма...\n")

334

335 app.mainloop()

336

337 if __name__ == "__main__":

338     start_gui()

339 # # Основной цикл

340 # def main():

341 #     while True:

342 #         choice = main_menu() # шоу меню

343 #         if choice == "1": # выбор загрузки правил из файла

344 #             rules = load_rules_from_file()

345 #             elif choice == "2": # пользователь выбрал ручной ввод правил

346 #                 rules = input_rules()

347 #             elif choice == "3": # выход

348 #                 print(" Завершение работы.")

349 #                 break

350 #             else: # некорректный ввод

351 #                 print(" Неверный выбор.\n")

352 #                 continue

353

354 #         # проверка: есть ли правила

355 #         if not rules:

356 #             print(" Нет загруженных правил.")

357 #             continue

```

```
358
359     #         text = input("\nВведите исходную строку: ").strip()
360
361     #         anim_choice = input("Включить анимацию (y/n)? ").lower() == "y"
362
363     #         #запуск
364     #         system = MarkovSystem(rules, animation=anim_choice)
365     #         result = system.run(text)
366
367     #         print(f"\n□n Результат: {result}\n")
368     #         input("Нажмите Enter, чтобы вернуться в меню...")
369
370
371     # if __name__ == "__main__":
372     #     main()
```